



---

# ROADSENSE AI

---

Road Damage Detection System



DECEMBER 10, 2025

**GROUP 5**

NUMAAN BAIG

ANMOL SHRESHTA

ABDALLA MOHAMED ABDALLA MOHAMED

JATURAPUT JONGSUBCHAROEN

RAJ PATEL

DANIELA SOFIA DIAZ PRADO

JUAN DAVID BARRERO GUERRERO

# Roadsense AI – Road Issues Detection System

## Technical Documentation

### 1. Project Overview

Roadsense AI is a road-issues detection system that uses a deep learning image classifier and a lightweight web backend to identify different types of road problems from images. The system is designed for municipal authorities or road-maintenance teams who need a fast way to triage citizen reports and prioritize repairs.

- Key capabilities include:
- Automated classification of road-related issues from uploaded images.
- REST API for prediction, chatbot interaction, and document-aware Q&A (RAG).
- Local LLM integration via Ollama for explanations and conversational support.
- Modular ML pipeline for data preprocessing, model training, and model serving.

### 2. System Architecture

The solution is structured as a set of clear components that communicate via file I/O and HTTP APIs:

- Data & Training Pipeline (Jupyter notebooks – Phases 2, 3, and 4).
- Model Serving Layer (TensorFlow EfficientNet model + `model_service.py`).
- Web Backend API (Flask app in `app.py` with CORS enabled).
- Local LLM Service (`ollama_service.py` calling a llama3.1 model hosted by Ollama).
- Retrieval-Augmented Generation (`rag_service.py` with FAISS + SentenceTransformer for document Q&A).

Typical request flow for prediction is:

- 1) The frontend sends an image to `/api/predict`.
- 2) `app.py` forwards the file to `model_service.predict_image`.
- 3) The EfficientNet model returns the predicted issue class and confidence.
- 4) The backend returns a JSON response to the frontend.

### 3. Dataset and Preprocessing (Phase 2)

Phase 2 (`phase2_preprocessing.ipynb`) prepares the image dataset and produces the CSV splits that are used in later phases.

- Main steps:
- Scan a root `DATA_DIR` where each sub-folder name corresponds to a road-issue class.
- Collect full file paths and derive the label from the parent folder name.
- Build a pandas DataFrame with columns `filepath` and `label`.

- Perform a stratified split into Train (70%), Validation (15%), and Test (15%) using `train_test_split`.
- Visualize class distribution in the training set to verify balance across labels.
- Create a `label_to_index` dictionary mapping each string label to an integer ID, plus the reverse `index_to_label`.
- Add a `label_idx` column to train, validation, and test DataFrames using the mapping.
- Save `train_split.csv`, `val_split.csv`, and `test_split.csv` into the `ml/data` directory for later phases.

This phase ensures that every image has a consistent integer label and that all three splits share the same label mapping. It also confirms that class balance is preserved through stratified sampling.

#### 4. TensorFlow Data Pipeline (Phase 3)

Phase 3 (`phase3_data_pipeline.ipynb`) converts the CSV splits into optimized `tf.data` pipelines that can be reused in model training.

- Key elements:
- Read `train_split.csv`, `val_split.csv`, and `test_split.csv` from a `SPLIT_DIR` location.
- Create a `path` column for each split, mirroring the `filepath` column.
- Define image parameters such as `IMG_SIZE = 224` and `BATCH_SIZE = 32`, and use `AUTOTUNE` for performance.
- Implement `load_image(path, label)` to read JPEG files, decode them as RGB, resize to `224×224`, and cast them to `float32` tensors.
- Define a `data_augmentation` Sequential model with `RandomFlip` and `RandomRotation`.
- Wrap augmentation in an `augment(img, label)` helper so that augmentation is applied only on the images.
- Build initial datasets using `tf.data.Dataset.from_tensor_slices((path, label_idx))` for train, validation, and test.
- Create final pipelines that shuffle, map `load_image`, map `augment` (for training only), batch, and prefetch using `AUTOTUNE`.

At the end of this phase, `train_ds`, `val_ds`, and `test_ds` are ready and can be passed directly into `model.fit` for training.

#### 5. Model Training with EfficientNet (Phase 4)

Phase 4 (`phase4_train_efficientnet.ipynb`) implements the full training loop for a transfer-learning classifier based on `EfficientNetB0`.

- Core steps:
- Load the CSV splits again and recreate `path` columns pointing to image files.
- Re-define `IMG_SIZE`, `BATCH_SIZE`, `AUTOTUNE`, and the same `load_image` and `augment` functions used in Phase 3.

- Compute class weights with `sklearn.utils.compute_class_weight` to address class imbalance.
- Create an EfficientNetB0 `base_model` with ImageNet weights, `include_top=False`, and `input_shape=(224, 224, 3)`.
- Freeze `base_model.trainable = False` for the warm-up phase.
- Build a custom classification head: `GlobalAveragePooling2D` → `Dropout(0.3)` → `Dense(num_classes, softmax)`.
- Construct the full model = `Model(inputs, outputs)` and compile with Adam (`learning_rate=1e-3`), `sparse_categorical_crossentropy` loss, and accuracy metrics.
- Define callbacks: `EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)` and `ReduceLROnPlateau(monitor='val_loss', patience=2, factor=0.5, min_lr=1e-6)`.
- Run Phase 1 training (`history1`) for around 10 epochs using class weights and the callbacks.
- Unfreeze the EfficientNet backbone for fine-tuning, optionally keeping the first ~200 layers frozen, and recompile with a lower learning rate (e.g., `1e-4`).
- Run Phase 2 training (`history2`) for additional epochs, again using class weights and callbacks.
- Evaluate the final model on `test_ds` to obtain final test accuracy and loss.
- Generate predictions on the test set to compute a confusion matrix and classification report.
- Plot accuracy and loss curves for both warm-up and fine-tuning phases using a helper function `plot_history`.
- Visualize sample predictions and misclassified examples to qualitatively inspect model performance.
- Save the trained model as `efficientnet_best_model.keras` and reload it to verify the saved artifact.

The resulting model artifact is later loaded by the production inference code in `model_service.py`.

## 6. Model Serving Layer (`model_service.py`)

The `model_service` module encapsulates all TensorFlow-related logic required at inference time. It is responsible for loading the trained EfficientNet model once at process start and exposing a simple Python function used by the Flask API.

- Responsibilities:
- Define `MODEL_PATH` pointing to `efficientnet_best_model.keras`.
- Load the Keras model using `tf.keras.models.load_model` at import time so the cost is paid only once.
- Define `CLASS_NAMES` as a list of human-readable labels (e.g., 'Pothole Issues', 'Damaged Road issues', 'Broken Road Sign Issues', etc.).

- Implement `preprocess(image_bytes)` that: opens bytes as a PIL image, converts to RGB, resizes to 224×224, converts to a float32 NumPy array, applies EfficientNet `preprocess_input`, and adds a batch dimension.
- Implement `predict_image(file)` that: reads the uploaded file bytes, calls `preprocess`, runs `model.predict`, extracts the highest-probability class index, and returns a dictionary with `class_name` and `confidence`.

This design keeps prediction logic isolated from Flask and makes it easy to reuse the same function in other contexts such as batch scoring scripts or unit tests.

## 7. Web Backend API (`app.py`)

The web backend is implemented as a Flask application with CORS enabled so that it can be called from a JavaScript frontend. The app exposes health, prediction, chatbot, and RAG endpoints.

- Endpoints:
- GET `/api/health` – returns a simple JSON status object. Used by the frontend or monitoring tools to check that the backend is running.
- POST `/api/predict` – expects a multipart/form-data request with an image file under the key 'image'. The endpoint calls `model_service.predict_image` and returns the predicted class and confidence as JSON.
- POST `/api/chat` – expects a JSON body with a 'message' field. The message is forwarded to `chat_with_ollama` and the AI response is returned as JSON.
- POST `/api/rag/upload` – expects a file upload under the key 'file'. The file is passed to `load_knowledge_from_file`, which indexes its content for later retrieval. The endpoint returns a status and information such as the number of chunks loaded.
- POST `/api/rag/ask` – expects a JSON body with a 'question' field. The question is routed to `chat_with_ollama_rag`, which performs retrieval-augmented generation based on the last uploaded document and returns an answer.

Error handling is performed for missing files or empty questions, returning HTTP 400 responses with descriptive error messages.

## 8. Local LLM Service (`ollama_service.py`)

The `ollama_service` module provides a thin wrapper over a locally running Ollama server. It is used both for the generic chatbot endpoint and as the language model in the RAG component.

- Behaviour:
- Define `OLLAMA_URL` pointing to `http://localhost:11434/api/generate`.
- Implement `chat_with_ollama(message)` which sends a POST request with `model='llama3.1'`, `prompt=message`, and `stream=False`.
- Parse the JSON response and return the 'response' text if available, or a fallback message if not.

- Catch exceptions and return an informative error string if the Ollama server is unavailable.

## 9. Retrieval-Augmented Generation Service (rag\_service.py)

The rag\_service module turns uploaded documents into a searchable knowledge base that can be queried through the RAG endpoints. It uses SentenceTransformer embeddings and a FAISS index to retrieve relevant chunks before calling the Ollama LLM.

- Key components:
- Environment configuration to disable TensorFlow, JAX/Flax, and unnecessary tokeniser parallelism inside the transformers stack.
- Global constants for OLLAMA\_URL and an UPLOAD\_DIR where incoming files are stored on disk.
- An embedding model based on SentenceTransformer('all-MiniLM-L6-v2') that converts text chunks into dense vectors.
- In-memory global variables DOCS (list of text chunks) and INDEX (FAISS index) that hold the current uploaded document.
- \_extract\_text\_from\_file(file\_path) that can read TXT, PDF (via PyPDF2.PdfReader), and DOCX (via docx2txt).
- \_chunk\_text(text, chunk\_size=800) that splits long documents into manageable chunks while preserving word boundaries.
- load\_knowledge\_from\_file(file\_storage) which saves the uploaded file, extracts text, chunks it, computes embeddings, builds a FAISS IndexFlatL2 index, and returns status information including the number of chunks loaded.
- \_retrieve\_context(query, top\_k=5) which embeds the query and returns the top-k most similar chunks from DOCS.
- chat\_with\_ollama\_rag(question) which: retrieves context, builds a prompt instructing the model to answer only using the provided context (or summarize it), and calls Ollama with a low-temperature setting for more deterministic answers.

If the retrieved context is insufficient, the prompt instructs the LLM to respond with a fixed fallback sentence, making the system safer and more predictable.

## 10. Running the System (High-Level Guide)

At a high level, the steps to run the complete system for demonstration are:

- Prepare the dataset folders with one sub-folder per road-issue class and place all images accordingly.
- Execute Phase 2, Phase 3, and Phase 4 notebooks in order to generate CSV splits, build the data pipeline, and train the EfficientNet model, saving efficientnet\_best\_model.keras.
- Ensure the model path in model\_service.py points to the saved model file.
- Install Python dependencies for the backend (Flask, TensorFlow, sentence-transformers, faiss, PyPDF2, docx2txt, requests, etc.).
- Start the Ollama server locally and pull the llama3.1 model if necessary.

- Run app.py to start the Flask backend (by default on port 5000).
- Use a frontend or tools such as Postman to call /api/predict, /api/chat, /api/rag/upload, and /api/rag/ask.

This documentation covers the current codebase (data pipeline, model training, and backend services). Additional components such as the frontend can be documented in a separate section using the same structure.